COMP214 — Advanced Database Concepts

Term Project – prof.Sarajlic

Group project · Maximum 4 students per group · Worth 15% of the final grade

Item	Detail
Due date	___End of W12_____ (submitted through the Dropbox on the LMS)
Presentation date	____W13 lab_____ (all group members must be present)
Group size	Maximum 4 students. Groups must be registered with the instructor by _W12_____.
Weight	15% of the final grade (see Section 3 for the mark breakdown).

1. Purpose

This project asks you to apply everything covered in the course and prior database intro to a working database application of your own design. On completion you will have:

- Applied comprehensive database knowledge to a practical problem of your choosing.
- Designed and implemented a relational schema with appropriate constraints.
- Built and tested sequences, indexes, triggers, procedures, functions and packages in PL/SQL.
- Written exception handlers that deal with real failure conditions, not just the happy path.
- Produced evidence that your own group designed, executed and verified the work.

2. The task

Create the database back end and a working front end (for bonus 5%) for an application of your choice, using Oracle Database. Examples: an e-commerce store, an airline booking site, an insurance quotation calculator, a clinic appointment system, a library, a food-delivery service… Choose a domain you can describe in one paragraph and scope it so that five to ten tables are enough to represent it properly.

You may build on the web application from COMP229, or start an independent project. The work covers interface design, database design and implementation, data creation, testing, documentation and presentation.

2.1 Front-end technology

Any language or framework that connects to your Oracle schema and calls your stored procedures, functions and packages is acceptable — Oracle APEX, Java/JDBC, Python, PHP, .NET, Node.js. Oracle APEX is recommended if your group has no strong preference, because it removes the connection plumbing and lets you spend your time on the database logic.

Required of every front end

The front end must call your PL/SQL programming units. A front end that issues only raw SELECT and INSERT statements against tables, bypassing your procedures, functions and packages, does not satisfy this requirement and will lose the interface bonus marks.

3. Mark breakdown

The project is worth 15 marks, equal to 15% of the final grade. Plus 5% bonus as described. Marks are distributed as follows.

A. Script file(s) — 10 marks

Component	Requirement	Marks
Tables & constraints	Minimum 5, maximum 10 tables. Must include data tables, lookup/reference tables, and at least one audit/log table. Primary keys, foreign keys, NOT NULL, UNIQUE and CHECK constraints as appropriate.	2.0
Sequences	At least one sequence, used both to insert sample data and to update table(s).	1.0
Indexes	At least two indexes on columns that your application actually searches on.	0.5
Triggers	At least two, serving different purposes (see 4.4).	2.0
Procedures	At least two, with cursors and exception sections (see 4.5).	2.0
Functions	At least two, with exception sections (see 4.6).	1.0
Packages	At least one package with spec and body, wrapping two procedures and two functions (see 4.7).	1.5
	Subtotal	10.0

B. Front-end interface and presentation — 3 marks

Component	Requirement	Marks
Working interface	The front end wireframe matches, relates to your schema, and exercises your programming units. Bonus 5%	1.5 +5.0
Live demonstration	The group demonstrates the application and answers questions on the code (see Section 8).	1.5
	Subtotal	3.0+5.0

C. Project document — 2 marks

Component	Requirement	Marks
Report	Group members, problem domain description, ER diagram, Wireframe of front end and the execution evidence described in Section 6.	2.0
	Subtotal	2.0

Total	Marks
Term project	15.0

4. Contents of your database

Everything in this section must be created by your script file(s). Each requirement below states a minimum. Meeting a minimum with two near-identical objects will not earn full marks — the objects must differ in purpose, as described.

4.1 Tables, constraints and data

- Minimum 5, maximum 10 tables.
- Include at least one lookup/reference table and at least one audit or log table that is written to by a trigger.
- Primary key on every table. Foreign keys wherever a relationship exists. Add NOT NULL, UNIQUE and CHECK constraints where the business rules call for them.
- At least 10 INSERT statements for each data table, and at least 5 rows for each lookup/reference table.
- Data must be meaningful for your domain. Rows such as "Customer1, Customer2, Customer3" or "aaa@aaa.com" will lose marks.

4.2 Sequences

- Create at least one sequence.
- Use it to insert sample data.
- Use it to update table(s).

4.3 Indexes

- Create at least two indexes.
- Place them on the columns your application searches on most often, and say in your report which search each index supports.

4.4 Triggers — at least two, with different purposes

Your two triggers must not both do the same kind of work. Include, at minimum:

- **One audit/logging trigger** — writes to your audit/log table when a row is inserted, updated or deleted.
- **One business-rule trigger** — enforces a rule that a constraint cannot express (for example, blocking an order when stock is insufficient, or maintaining a running total on a parent row).

4.5 Procedures — at least two

- At least one procedure must use an explicit cursor and process multiple rows.
- At least one procedure must perform DML with validation before the change is applied.
- Both procedures must have an exception section that handles named exceptions relevant to what the procedure does. WHEN OTHERS THEN NULL is not an exception handler and will be marked as zero for that requirement.
- Where a procedure signals a business error, raise it with RAISE_APPLICATION_ERROR and a message a user could understand.

4.6 Functions — at least two

- At least one function must return a scalar value derived from a query (a count, total, average or status).
- At least one function must be called from somewhere else in your own code — a trigger, a procedure, a view or a CHECK-style validation.
- Every execution path must reach a RETURN. Both functions must have an exception section.

4.7 Packages — at least one

- Submit both the package specification and the package body.
- The package must expose at least two procedures and at least two functions (these may be ones you have already written).
- Include at least one global (public) variable or constant declared in the spec, and at least one private variable or private subprogram declared only in the body that has to be used in testing and running, executing.
- Use %TYPE and/or %ROWTYPE wherever a variable mirrors a column or a row.
- Packaged subprograms need exception handling on the same terms as the standalone ones.

5. Script file standards

Your script(s) are executed by me on a clean schema. They must run start to finish without manual intervention.

5.1 Re-runnable

Scripts must be safe to run more than once. Begin with a drop section so that a failed run can be reset and retried:

```
DROP TABLE order_items CASCADE CONSTRAINTS;
```

```
DROP TABLE orders      CASCADE CONSTRAINTS;
DROP SEQUENCE order_seq;
-- ... one line per object, children before parents
```

A script that fails on its second run because objects already exist will lose marks.

5.2 Header block

Every script file starts with a comment block identifying the group and members and the file name.

5.3 Order of objects

Order the script so it executes cleanly: drops, tables and constraints, sequences, indexes, sample data, functions, procedures, packages, triggers, then the demonstration script (Section 5.4).

5.4 Demonstration script

Include a separate demonstration script that exercises every programming unit you have written. For each object it must show both outcomes:

- **The success path** — the call works and produces the correct result.
- **The failure path** — an input that triggers your exception handler, and the handled error it produces.

Begin the demonstration script with the following two lines, so every screenshot carries your session identity and the date:

```
SET SERVEROUTPUT ON;
SELECT USER AS connected_as, SYSDATE AS run_on FROM dual;
```

6. Evidence of execution

The project document must show that your group actually ran this code. Screenshots are the record of that. The standards below exist so that evidence is verifiable; screenshots that do not meet them will not be counted toward the mark of that object type. As if that was just a copy not created and debugged / tested by the group / member.

6.1 What each screenshot must show

- The full application window — not a cropped output pane. The connected user, the SQL Developer session, the statement, and its result must all be visible in the same image.
- Code and output in the same frame. A screenshot of code in one image and output in another will not be accepted as evidence of execution.
- A visible session timestamp. Running the :
- SET SERVEROUTPUT ON;SELECT USER AS connected_as, SYSDATE AS run_on FROM dual;
- Legible text. Take screenshots at full resolution; do not shrink them below readable size when placing them in the document.

6.2 Pasted text alongside every screenshot

Under each screenshot, paste the same output as selectable text — the statement you ran and the result it returned. The document must contain both the image and the text.

6.3 Captions

Caption every screenshot with three things:

- The requirement it satisfies (for example, "4.5 — procedure with explicit cursor, success path").
- The name of the object being demonstrated.
- The group member who ran it.

6.4 Required evidence by object type

At a minimum, your document must contain the following.

Object	Required screenshots
Tables	The CREATE statements executing successfully, plus a SELECT showing the populated rows of each table (row count visible).
Constraints	One deliberate violation per constraint type you implemented — the ORA error returned when the bad data is rejected.
Sequence	The sequence in use during an insert, and in use during an update, with the resulting rows.
Indexes	The CREATE INDEX statements, plus the search query each index supports.
Triggers	Three screenshots per trigger: (1) the log/target table before, (2) the DML that fires the trigger, (3) the same table after, showing the row the trigger wrote or the action it blocked.
Procedures	Two per procedure: one successful call with its output, and one call that raises the handled exception, showing the handled message.
Functions	Two per function: one call returning the correct value, and one call that raises the handled exception.
Package	The spec and body compiling without errors, plus at least one call to each packaged subprogram through the package name and utilizing global variable.
Front end	Screenshots of each screen of your interface, including one showing a result that came back from a stored procedure or function.

7. Project document — required structure

Submit as MS Word or PDF, in this order:

- Title page: project name, group name , course and section, submission date.
- Group members: full name and student ID for each member.
- **Contribution statement**: a short table listing what each member built — which tables, which programming units, which parts of the front end, who ran which demonstrations.

- Problem domain description: one to two paragraphs explaining what the application does and who uses it.
- ER diagram: all tables, keys and relationships, legible at full page size.
- Wireframe of front end.
- Table descriptions: the purpose of each table and the constraints on it.
- Programming unit descriptions: for each trigger, procedure, function and package — what it does, what exceptions it handles, and where it is called from.
- Execution evidence: the screenshots and pasted output required by Section 6, in the order of Section 4.
- Front-end screenshots with a short explanation of each screen.

8. Presentation and live verification

The presentation is where execution is verified. Screenshots are the record; the demonstration is the proof.

- All group members must attend and speak.
- **Any member may be asked to run any part of the script live and explain what it does.** This includes code written by another member of the group.
- Expect questions.
- The front-end and demonstration (for bonus) marks depend on this session. A member who cannot explain the work they are credited with in the contribution statement will not receive those marks.

9. Use of generative AI — not permitted

Specifically, you may not use an AI tool to:

- Generate, complete or rewrite any SQL or PL/SQL code, in whole or in part.
- Design your schema, suggest tables or relationships, or produce your ER diagram.
- Generate sample data for your tables.
- Write, edit, summarise or proofread any part of the project document.
- Explain, debug or fix your code.
- Produce your front-end code or interface layout.

You may use: the course notes and lecture material, the Oracle documentation, your textbook, the Oracle SQL Developer built-in help, and discussion with your own group members. If you use a code fragment from documentation or a textbook, cite it in a comment in the script.

9. Submission

9.1 What to submit

Submit through the Dropbox on the LMS. One submission per group, made by any one member.

#	File	Naming
1	Schema and objects script	GroupName_ProjectName_TermProject_Schema.sql
2	Demonstration script	GroupName_ProjectName_TermProject_Demo.sql
3	Project document (Word or PDF)	GroupName_ProjectName_TermProject_Report.docx
4	Front-end source code (zipped) for bonus	GroupName_ProjectName_TermProject_FrontEnd.zip

Example: FlowerShop_OnlineFlorist_TermProject_Schema.sql

Use .sql for script files. Do not use "#" or spaces in file names.

9.2 Submission checklist for me

I will check each item before marking:

Item	Done
Script runs start to finish on a clean schema with no manual intervention	
Script can be run a second time without errors (drop section present)	
5–10 tables, including a lookup table and an audit/log table	
10+ rows per data table, 5+ rows per lookup table, meaningful values	
At least one sequence, used for both an insert and an update	
At least two indexes on searched columns	
Two triggers with different purposes (audit + business rule)	
Two procedures, one using an explicit cursor, both with exception sections	
Two functions, one called from elsewhere in your code, both with exception sections	
Package spec and body, 2 procedures + 2 functions, global and private constructs, %TYPE/%ROWTYPE	
Demonstration script shows success AND failure path for every programming unit	
Screenshots show full window, connected user, session timestamp, code and output together	
Pasted text output under every screenshot	
Every screenshot captioned with requirement, object name and group member	

Item	Done
Three screenshots per trigger (before / DML / after)	
ER diagram included and legible	
Contribution statement completed and initialled by each member	
Declaration of original work signed by every member	
All four files named per Section 9.1	